

Cache Strategies & Distributed Cache Design

Java Backend Interview Reference — Deep Dive

Cache Strategies — Read Patterns

Strategy	Flow	Pros	Cons	Use Case
Cache-Aside (Lazy Loading)	1. App checks cache 2. Miss: app loads DB 3. App writes to cache 4. Return data	Only cache what's needed Resilient: cache failure doesn't break app	Cache miss penalty Stale data possible Race condition on miss	General reads Product catalog User profiles
Read-Through	1. App reads cache 2. Miss: cache loads DB 3. Cache returns data to app	Cache handles DB fetch Consistent cache population	First request always misses Cache must understand data model	ORMs with L2 cache Hibernate 2nd level

Cache Strategies — Write Patterns

Strategy	Flow	Pros	Cons	Use Case
Write-Through	Write -> cache AND DB synchronously. Both always in sync.	Strong consistency No stale reads	Write latency doubles Caches unused data (write but never read)	Banking balance Inventory counts
Write-Behind (Write-Back)	Write -> cache first. DB updated asynchronously after delay (batch).	Low write latency Batch DB writes (efficient)	Data loss risk if cache crashes before flush Complexity	Shopping cart Analytics counters Like counts
Write-Around	Write directly to DB, bypass cache. Cache populated on next read miss.	Cache not polluted with write-once data Good for bulk loads	Next read always misses	Log ingestion Bulk data import
Refresh-Ahead	Cache pre-emptively refreshes expiring items before TTL expires.	No miss latency for hot items Smooth UX	Might refresh items not re-requested (waste)	Homepage widgets Top-N leaderboards Hot config data

Cache Problems & Solutions

Problem	What Happens	Fix
Cache Penetration	Requests for keys that NEVER exist in DB. Cache always misses; DB gets hammered.	Cache negative results (null) with short TTL (60s) Bloom filter at gateway to reject impossible keys
Cache Breakdown (Thundering Herd)	A single hot key expires. Many requests simultaneously miss, all hit DB at once.	Mutex/lock: first miss acquires lock, loads DB, others wait Staggered TTL (base + small random) Single-flight pattern (deduplicate concurrent loads)
Cache Avalanche	Many keys expire at same time (e.g., app restart populates with same TTL). Massive DB spike.	Random TTL jitter: $TTL = base + random(0, base \cdot 0.2)$ Warm cache before deploy (pre-population) Circuit breaker on DB to prevent cascade
Cache Pollution	Cache fills with rarely-accessed data, evicting hot items.	Use LRU/LFU eviction appropriate for access pattern Set explicit TTL on all keys Segment caches by data type
Stale Data	Cache has old data after DB update. Reads get outdated info.	Short TTL for volatile data Event-driven invalidation (publish event on update) Write-through pattern
Inconsistency	Cache and DB diverge — cache update fails after DB write.	Cache-aside with short TTL (accept eventual consistency) Distributed transaction (complex) Outbox pattern + event-driven invalidation

Designing a Distributed Cache System

Requirements (Example: Design Redis-like Distributed Cache)

Functional	Non-Functional
GET(key) -> value SET(key, value, ttl) DELETE(key) Support String, List, Hash, Set, Sorted Set types	Low latency (<1ms p99) High availability (99.99%) Horizontal scalability to N nodes Tunable consistency Partition tolerance

Architecture Components

Component	Role	Detail
Client Library	Routes requests to correct shard	Consistent hash ring; handles retries and failover
Cache Nodes	Store key-value data in memory	N nodes, each owning a partition of key space
Consistent Hashing Ring	Distribute keys across nodes	Virtual nodes (vnodes) prevent hotspots on node add/remove
Coordinator / Proxy	Optional: centralizes routing	Twemproxy, Envoy sidecar, Redis Cluster bus
Replication	Each shard has replicas (primary + 1-2 replicas)	Async replication for low latency; sync for strong consistency
Leader Election	Detect primary failure, elect replica as new primary	Raft (etcd), Sentinel (Redis), or ZooKeeper
Monitoring	Track hit rate, memory, evictions, latency	Prometheus exporter + Grafana dashboard

Consistent Hashing

```
// Simplified consistent hash ring in Java
public class ConsistentHashRing {
    private final TreeMap<Long, CacheNode> ring = new TreeMap<>();
    private final int virtualNodes;

    public void addNode(CacheNode node) {
        for (int i = 0; i < virtualNodes; i++) {
            long hash = hash(node.getId() + "#" + i);
            ring.put(hash, node);
        }
    }

    public CacheNode getNode(String key) {
        long hash = hash(key);
        Map.Entry<Long, CacheNode> entry = ring.ceilingEntry(hash);
        return entry != null ? entry.getValue() : ring.firstEntry().getValue();
    }

    private long hash(String key) {
        // Use MurmurHash or MD5 for good distribution
        return Math.abs(key.hashCode()) % Long.MAX_VALUE;
    }
}
```

Data Partitioning Strategies

Strategy	How	Pros	Cons
Hash Partitioning	hash(key) % N determines node	Even distribution Simple	Node add/remove requires rehash of all keys
Consistent Hashing	Key maps to position on ring; nearest node owns it	Only K/N keys move on node change	Potential hotspots without vnodes
Range Partitioning	Keys A-M on node 1, N-Z on node 2	Range queries efficient	Hotspots if key space not uniform
Directory-based	Lookup table maps keys to nodes	Flexible; easy rebalancing	Directory is single point of failure; latency

Replication & Consistency

Model	How	Consistency	Availability
Primary-Replica (async)	Writes go to primary; replicated asynchronously	Eventual	High — replica can serve reads
Primary-Replica (sync)	Write ACK only after all replicas confirm	Strong	Lower — write blocked if replica down
Quorum (W+R>N)	N=3 replicas, W=2 writes, R=2 reads	Tunable	Fault tolerant up to (N-W) failures
Multi-primary	Any node accepts writes, resolve conflicts	Eventual	Highest

Eviction Policies

Policy	Description	Best For
LRU (Least Recently Used)	Evict key not accessed for longest time	General workloads; temporal locality
LFU (Least Frequently Used)	Evict key with fewest accesses over time	Hot/cold data distinction; skewed access
ARC (Adaptive Replacement Cache)	Self-tuning between LRU and LFU	Workloads that shift between access patterns
TTL / Expiry	Evict when time-to-live expires	Sessions, tokens, rate limit windows
Random	Evict random key (simple but suboptimal)	Only when overhead of tracking is too high
No-Eviction	Return error when memory full; no eviction	Caches where data loss unacceptable; must use TTL

Redis Architecture Deep Dive

Mode	Topology	Use Case
Standalone	Single Redis instance; no HA	Dev/test; simple caching
Sentinel	1 primary + N replicas + Sentinel monitors for failover. Quorum of Sentinels vote to elect new primary	HA for single shard; most common for moderate scale
Cluster	16,384 hash slots distributed across N primary nodes, each with replicas. Gossip protocol for node discovery	Horizontal scale; >~100GB data or high write throughput

Redis Cluster — Key Distribution

```
// All keys in one transaction/pipeline MUST be on same node // Use hash tags: {user:123}:cart and
{user:123}:session go to same slot redisTemplate.opsForHash().put("{user:123}:cart", "item1", "2");
redisTemplate.opsForHash().put("{user:123}:session", "lastSeen", now); // CROSS-SLOT operations not supported
in cluster: // MGET {key1} {key2} -- ERROR if different slots
```

Spring Boot — Redis Cache Setup

```
# application.yml spring: cache: type: redis data: redis: host: localhost port: 6379 lettuce: pool:
max-active: 20 max-idle: 10 // Config @Configuration @EnableCaching public class CacheConfig { @Bean public
RedisCacheManager cacheManager(RedisConnectionFactory factory) { RedisCacheConfiguration config =
RedisCacheConfiguration.defaultCacheConfig() .entryTtl(Duration.ofMinutes(30)) .
serializeValuesWith(RedisSerializationContext.SerializationPair .fromSerializer(new
GenericJackson2JsonRedisSerializer())); return RedisCacheManager.builder(factory).cacheDefaults(config).build();
} } // Usage @Cacheable(value = "products", key = "#id", unless = "#result == null") public Product
getProduct(Long id) { return repo.findById(id).orElse(null); } @CacheEvict(value = "products", key = "
#product.id") public Product updateProduct(Product product) { return repo.save(product); } @CachePut(value = "
products", key = "#result.id") // always update cache public Product createProduct(Product product) { return
repo.save(product); }
```

Interview Q&A;

Q1: What is consistent hashing and why is it used in distributed caches?

Consistent hashing maps both keys and nodes onto a circular ring using a hash function. Each key is assigned to the nearest node clockwise on the ring. When a node is added or removed, only K/N keys (K=total keys, N=nodes) need to be remapped — unlike modulo hashing where nearly all keys move. Virtual nodes (vnodes) are added to improve distribution uniformity.

Q2: Explain the Thundering Herd problem and how to prevent it.

When a hot cached key expires, many concurrent requests simultaneously miss the cache and all hit the database. Solutions: (1) Mutex/lock — first miss acquires a lock and loads DB; others wait for cache to be populated; (2) Stale-while-revalidate — serve stale data while asynchronously refreshing; (3) Probabilistic early expiration — randomly extend TTL for some requests to spread reloads; (4) Single-flight (singleflight.Group in Go / custom in Java) — deduplicate identical in-flight requests.

Q3: How do you handle cache invalidation across microservices?

Publish a domain event (e.g., via Kafka) when data changes. Each service that caches that data subscribes and evicts/updates its cache on the event. This is event-driven invalidation. Alternative: short TTLs (accept eventual consistency). Avoid direct cache-to-cache calls. For critical consistency, use the Outbox pattern to ensure the event is always published even if the service crashes after DB write.

Q4: What is the difference between LRU and LFU eviction?

LRU (Least Recently Used) evicts the key that hasn't been accessed for the longest time. It favors temporal recency. LFU (Least Frequently Used) evicts the key with the fewest total accesses. LFU is better for workloads with persistent hot keys (popular products) while LRU handles temporal bursts better (news article trending for a day). Redis uses LRU by default; LFU is available via `maxmemory-policy allkeys-lfu`.

Q5: What is the quorum model in a distributed cache?

With N replicas, require W write confirmations and R read responses such that $W + R > N$. This guarantees at least one node has the latest value. Example: $N=3, W=2, R=2$. Can tolerate $N-W=1$ write failure and $N-R=1$ read failure. Tunable: $W=1, R=3$ gives fast writes, strong reads; $W=3, R=1$ gives slow writes, fast reads; $W=2, R=2$ is balanced.

Q6: How would you design a rate limiter using Redis?

Use Redis sorted sets or token bucket with atomic Lua scripts. Sliding window: `ZADD key timestamp timestamp`; `ZREMRANGEBYSCORE key 0 (now-window)`; `ZCARD key` returns current count. Or Fixed window: `SET key:window count EX ttl`; `INCR key`. Always use atomic Lua scripts for check-and-increment to prevent race conditions. Cluster deployments: use hash tags to ensure key stays on one node.

Q7: How do you prevent cache stampede on application startup?

Pre-warm the cache before going live: run a startup script or scheduled job that loads the N most frequent keys into cache. Use a canary deployment that pre-warms before routing production traffic. Alternatively, stagger TTLs: don't set everything to expire at the same time. In Kubernetes: use an init container or startup probe that pre-populates Redis before the pod is marked ready.

Q8: What is the difference between Redis Sentinel and Redis Cluster?

Sentinel monitors a single-shard Redis deployment (1 primary + N replicas) and handles automatic failover — if the primary dies, Sentinels vote to promote a replica. It does NOT scale writes across multiple nodes. Redis Cluster shards data across multiple primary nodes (each with replicas) using 16,384 hash slots, providing both horizontal scalability and HA. Use Sentinel for HA on a single dataset; use Cluster when dataset exceeds ~50GB or write throughput exceeds single-node capacity.

Q9: How do you handle cache consistency in a multi-region setup?

Options: (1) Single global cache (Redis in one region) — simplest but high latency for remote regions; (2) Read replicas per region with eventual consistency — low read latency, stale risk; (3) Write to all regional caches on update (fanout) — complex, risk of partial failure; (4) Each region has local cache + short TTL — accept eventual consistency; (5) CRDTs for conflict-free merge (complex). For most apps, option 2 or 4 with short TTL is pragmatic.